

# ISIS-2111 PLE

## Proyecto 4 - VeriLang en Kotlin

### Informe final corregido y verificado

Entrega: proyecto\_rascal

Autor: Roni Lopez

Fecha de verificación: 30 de mayo de 2026

|                                  |                                                                                                           |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Proyecto</b>                  | VeriLang reutilizado desde Rascal y ejecutado desde una aplicación Kotlin/Compose.                        |
| <b>Objetivo principal</b>        | Mantener equivalencia funcional con Proyecto 3 y exponer parser, checker y generator desde Kotlin.        |
| <b>Runtime Kotlin</b>            | Aplicación desktop en kotlin-app con Gradle 8.7, Compose Desktop y paquete verilang.                      |
| <b>Runtime Rascal</b>            | rascal-shell-stable.jar incluido en la entrega; TypePal vendorizado en src/main/rascal/analysis/typepal/. |
| <b>Resultado de verificación</b> | Rascal y Kotlin pasaron las pruebas obligatorias, incluyendo el caso inválido Undefined space Person.     |

## Contenido del informe

1. Alcance y requisitos del Proyecto 4
2. Requisitos heredados del Proyecto 3
3. Estructura real del entregable
4. Arquitectura general de la solución
5. Implementación Rascal: gramática, AST, parser, generator y checker
6. Integración Kotlin: RunnerJson, LangService, RunResult y UI
7. Correcciones frente a la rúbrica anterior
8. Evidencia de ejecución y pruebas
9. Matriz de verificación final
10. Instrucciones de ejecución en Windows
11. Conclusión

## 1. Alcance y requisitos del Proyecto 4

Este documento reemplaza la versión anterior del informe y corrige inconsistencias de presentación. La solución documentada corresponde al proyecto real contenido en la carpeta proyecto\_rascal, no a una plantilla genérica.

El Proyecto 4 solicita reutilizar VeriLang del proyecto anterior, pero ejecutarlo desde el runtime de Kotlin. Además, exige que la solución sea funcionalmente equivalente al Proyecto 3 y que se entregue un archivo ZIP con la solución y el documento.

| Requisito explícito del Proyecto 4            | Cómo se cumple en esta entrega                                                                                                     |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Reutilizar VeriLang del proyecto anterior.    | Se conserva la implementación Rascal en src/main/rascal/: Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson.     |
| Apuntar a Kotlin, no a Java.                  | La carpeta kotlin-app contiene una app Kotlin/Compose Desktop con paquete verilang.                                                |
| Ser funcionalmente equivalente al Proyecto 3. | Kotlin invoca RunnerJson.rsc, y RunnerJson ejecuta parser, TypePal/checker y generator sobre archivos .vl.                         |
| Ejecutar desde el runtime de Kotlin.          | LangService.kt inicia java -jar rascal-shell-stable.jar desde Kotlin y lee el JSON de RunnerJson.                                  |
| Usar el helper/skeleton Kotlin como base.     | El skeleton fue adaptado: la UI ahora dice VeriLang, selecciona archivos .vl y muestra parseo, tipos, semántica, errores y salida. |
| Entregar ZIP con solución y documento.        | La estructura contiene META-INF/RASCAL.MF, pom.xml, src/main/rascal, instance, kotlin-app, rascal-shell-stable.jar y este informe. |

## 2. Requisitos heredados del Proyecto 3

Como el Proyecto 4 exige equivalencia funcional con Proyecto 3, la entrega mantiene los requisitos de parser, generación, TypePal, anotaciones de tipo y chequeo semántico.

| Requisito heredado                                 | Implementación                                                                                                  |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Parser para VeriLang.                              | Parser.rsc expone parseMainModule y se usa desde Main.rsc y RunnerJson.rsc.                                     |
| Generar código desde archivos .vl.                 | Generator.rsc convierte el CST a AST mediante Implode.rsc y luego reconstruye el programa VeriLang.             |
| Mostrar resultado en consola.                      | Main.rsc imprime el resultado generado cuando no hay errores.                                                   |
| Instalar TypePal.                                  | Checker.rsc extiende analysis::typepal::TypePal; TypePal está vendORIZADO en src/main/rascal/analysis/typepal/. |
| Agregar anotaciones de tipo para valores.          | Syntax.rsc y AST.rsc soportan int, bool, real, string, char y dominios de usuario.                              |
| Estructuras de datos con tipo dado por el usuario. | Space soporta typedSpace y typedSubspace: defspace Group :                                                      |

|                                                           |                                                                                                                     |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|                                                           | Person end.                                                                                                         |
| Verificar correspondencia de tipos.                       | Checker.rsc define tipos primitivos, userType, functionType, roles y validaciones con requireEqual.                 |
| Regla de existencia para elementos usados en estructuras. | useDomainIfUserDefined registra usos de dominios de usuario contra spaceId y reporta Undefined space si no existen. |

### 3. Estructura real del entregable

La carpeta principal del proyecto debe llamarse proyecto\_rascal porque el manifiesto Rascal declara Project-Name: proyecto\_rascal. Para evitar errores de ejecución, Rascal debe abrirse desde esa carpeta.

```
proyecto_rascal/
  META-INF/RASCAL.MF
  pom.xml
  rascal-shell-stable.jar
  README_ENTREGA_PROYECTO4.md
  src/main/rascal/
    Syntax.rsc
    AST.rsc
    Parser.rsc
    Implode.rsc
    Generator.rsc
    Checker.rsc
    Main.rsc
    RunnerJson.rsc
    analysis/typepal/...
  instance/
    spec1.vl
    spec2.vl
    spec3.vl
    spec_types.vl
    spec_typed_space_good.vl
    spec_typed_space_quantifier_good.vl
    errors/...
  kotlin-app/
    build.gradle.kts
    settings.gradle.kts
    gradlew
    gradlew.bat
    gradle/wrapper/gradle-wrapper.jar
    src/main/kotlin/verilang/...
```

| Elemento revisado         | Evidencia en el proyecto                                                                              | Estado |
|---------------------------|-------------------------------------------------------------------------------------------------------|--------|
| Manifiesto Rascal         | META-INF/RASCAL.MF contiene Project-Name: proyecto_rascal y Source: src/main/rascal.                  | Cumple |
| Módulos Rascal requeridos | Están Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson.                            | Cumple |
| App Kotlin                | Está en kotlin-app con Gradle wrapper, Compose Desktop y paquete verilang.                            | Cumple |
| Runtime Rascal            | rascal-shell-stable.jar está incluido para que Kotlin pueda invocar Rascal.                           | Cumple |
| Casos de prueba           | instance/ contiene casos válidos e inválidos, incluyendo spec_types.vl y spec_unknown_space_error.vl. | Cumple |

## 4. Arquitectura general de la solución

La arquitectura separa la definición del lenguaje en Rascal y la experiencia de ejecución en Kotlin. Kotlin no reimplementa la semántica; actúa como runtime/interfaz que delega a VeriLang en Rascal.

```
Usuario selecciona archivo .vl en la UI Kotlin
|
v
MainWindow.kt -> LangService.kt
|
v
java -Dfile.encoding=UTF-8 -jar rascal-shell-stable.jar
|
v
import RunnerJson; RunnerJson::main(["archivo.vl"])
|
v
RunnerJson.rsc -> Parser.rsc -> Checker.rsc/TypePal -> Generator.rsc
|
v
JSON: success, parseOk, typeCheckOk, semanticOk, errores, output
|
v
RunResult.kt -> MainWindow.kt muestra estados, errores y código generado
```

## 5. Implementación Rascal

### 5.1 Gramática concreta: Syntax.rsc

Syntax.rsc define la sintaxis concreta de VeriLang. La corrección importante frente a la rúbrica anterior es que Domain incluye tipos primitivos y tipos definidos por el usuario, y Space permite estructuras tipadas.

```
syntax Domain
  = boolDomain: 'bool'
  | intDomain: 'int'
  | realDomain: 'real'
  | stringDomain: 'string'
  | charDomain: 'char'
  | nameDomain: ID domainName
;

syntax Space
  = space: 'defspace' ID spaceName 'end'
  | typedSpace: 'defspace' ID spaceName ':' Domain elementType 'end'
  | subspace: 'defspace' ID subSpace '<' ID superSpace 'end'
  | typedSubspace: 'defspace' ID subSpace '<' ID superSpace ':' Domain elementType 'end'
;
```

La gramática también cubre variables tipadas, operadores, reglas, expresiones lógicas, relaciones, literales numéricos, booleanos, string, char y cuantificadores forall/exists/defer.

### 5.2 Sintaxis abstracta: AST.rsc

AST.rsc está alineado con la gramática. En particular, Space y Domain representan explícitamente espacios tipados y tipos definidos por usuario.

```
data Space
  = space(str spaceName)
  | typedSpace(str spaceName, Domain elementType)
  | subspace(str subSpace, str superSpace)
  | typedSubspace(str subSpace, str superSpace, Domain elementType)
```

```

;

data Domain
    = boolDomain()
    | intDomain()
    | realDomain()
    | stringDomain()
    | charDomain()
    | nameDomain(str domainName)
;

```

### 5.3 Parser e Implode

Parser.rsc parsea archivos .vl usando la gramática concreta. Implode.rsc transforma el árbol de parseo en el AST definido en AST.rsc. Ambos son usados por Generator.rsc y por el flujo principal.

### 5.4 Generator.rsc

Generator.rsc genera texto VeriLang a partir del AST. Cubre espacios normales, espacios tipados, subespacios, variables con tipos, operadores, reglas, expresiones, relaciones y cuantificadores.

```

str generate(typedSpace(str spaceName, Domain elementType)) {
    return "defspace <spaceName> : <generate(elementType)> end";
}

str generate(varDecl(str name, Domain domain)) = "<name> : <generate(domain)>";

str generate(quantExp(Quantifier q, str o1, str o2, TopExp body)) {
    return "(<generate(q)> <o1> in <o2> . <generate(body)>)";
}

```

### 5.5 Checker.rsc y TypePal

Checker.rsc extiende TypePal y define tipos, roles y reglas de colección/validación. No depende de una librería Maven externa de TypePal; las fuentes de TypePal se incluyen dentro del proyecto en src/main/rascal/analysis/typepal/.

```

extend analysis::typepal::TypePal;

data AType
    = boolType()
    | intType()
    | realType()
    | stringType()
    | charType()
    | userType(str name)
    | functionType(list[AType] signature)
;

data IdRole
    = variableId()
    | operatorId()
    | spaceId()
;

```

La regla useDomainIfUserDefined es la pieza que corrige el problema de existencia de tipos definidos por usuario. Cuando un dominio es userType, el checker exige que exista como spaceId.

```

void useDomainIfUserDefined(Domain d, Collector c) {
    if (domainToType(d) is userType) {
        c.use(d, {spaceId()});
    }
}

```

```
}
```

Para estructuras tipadas y cuantificadores, el checker registra el tipo de elemento de cada estructura. Esto evita el error anterior donde un cuantificador sobre Group trataba la variable como Group en vez de Person.

```
map[str, AType] typedSpaceElementTypes = ();

void registerTypedSpaces(Tree pt) {
    typedSpaceElementTypes = ();
    if (pt has top) { pt = pt.top; }
    visit(pt) {
        case (Space) `defspace <ID name> : <Domain elementType> end`: {
            typedSpaceElementTypes["<name>"] = domainToType(elementType);
        }
        case (Space) `defspace <ID sub> \< <ID super> : <Domain elementType> end`: {
            typedSpaceElementTypes["<sub>"] = domainToType(elementType);
        }
    }
}

AType quantifiedVariableType(str domainName) {
    if (domainName in typedSpaceElementTypes) {
        return typedSpaceElementTypes[domainName];
    }
    return userType(domainName);
}
```

## 5.6 Main.rsc

Main.rsc implementa el flujo principal para consola: parsea, chequea con TypePal, detiene la generación si hay errores y retorna 0 o 1 según el resultado.

| Caso              | Comportamiento de Main.rsc                                                     |
|-------------------|--------------------------------------------------------------------------------|
| Programa válido   | Parse OK, TypePal sin errores, genera código, imprime salida y retorna int: 0. |
| Programa inválido | Parse OK, TypePal reporta errores, no genera código y retorna int: 1.          |

## 5.7 RunnerJson.rsc

RunnerJson.rsc es el puente estable entre Kotlin y Rascal. Recibe la ruta del archivo, normaliza rutas de Windows, ejecuta parser/checker/generator y emite un único objeto JSON usable por Kotlin.

```
JSON producido por RunnerJson.rsc:
{
    "success": boolean,
    "module": "VeriLang",
    "parseOk": boolean,
    "typeCheckOk": boolean,
    "semanticOk": boolean,
    "typeErrors": [ ... ],
    "semanticErrors": [ ... ],
    "output": [ ... ],
    "error": "...",
    "codigoFormateado": "...",
    "resumen": "..."
}
```

## 6. Integración Kotlin

### 6.1 build.gradle.kts y settings.gradle.kts

La app Kotlin usa Gradle 8.7, Kotlin JVM 1.9.22, Compose Desktop 1.6.0, Kotlin serialization y coroutines. El nombre del proyecto es verilang-app y el paquete de la aplicación es verilang.

```
plugins {  
    kotlin("jvm") version "1.9.22"  
    id("org.jetbrains.compose") version "1.6.0"  
    kotlin("plugin.serialization") version "1.9.22"  
}  
  
mainClass = "verilang.MainKt"  
packageName = "VeriLang"
```

### 6.2 Main.kt

Main.kt abre una ventana Compose titulada VeriLang. Esto demuestra que la aplicación no quedó con textos genéricos del skeleton.

```
Window(  
    onCloseRequest = ::exitApplication,  
    title = "VeriLang",  
    state = rememberWindowState(width = 800.dp, height = 600.dp)  
) {  
    MainWindow()  
}
```

### 6.3 LangService.kt

LangService.kt es la integración crítica del Proyecto 4. No es una plantilla: localiza la raíz proyecto\_rascal, verifica rascal-shell-stable.jar, inicia Rascal como subprocesso, importa RunnerJson, ejecuta RunnerJson::main con la ruta seleccionada, extrae el JSON del stdout y lo decodifica.

```
val commands = buildString {  
    appendLine("import RunnerJson;")  
    appendLine("RunnerJson::main([$escapedPath]);")  
    appendLine(":quit")  
}  
  
val process = ProcessBuilder(  
    "java",  
    "-Dfile.encoding=UTF-8",  
    "-jar",  
    rascalJar.absolutePath  
)  
    .directory(projectRoot)  
    .redirectErrorStream(false)  
    .start()
```

### 6.4 RunResult.kt

RunResult.kt tiene los mismos campos que el JSON emitido por RunnerJson.rsc. Esto permite que la UI lea success, parseOk, typeCheckOk, semanticOk, errores, output, codigoFormateado y resumen sin transformaciones frágiles.

### 6.5 MainWindow.kt

MainWindow.kt permite seleccionar archivos .vl, correr VeriLang y mostrar estados de parseo, tipos y semántica. También muestra errores de tipos, errores semánticos, código generado y output.

```

StatusChip("Parse", r.parseOk, green, red)
StatusChip("Types", r.typeCheckOk, green, yellow)
StatusChip("Semántica", r.semanticOk, green, red)

if (r.typeErrors.isNotEmpty()) {
    SectionBox("Errores de tipos", r.typeErrors.joinToString("\n"), yellow)
}

if (r.codigoFormateado.isNotBlank()) {
    SectionBox("Código generado", r.codigoFormateado, Color(0xFF90CAF9))
}

```

## 7. Correcciones frente a la rúbrica anterior

La retroalimentación del Proyecto 3 indicaba tres puntos principales: soporte incompleto para tipos definidos por usuario, revisión insuficiente de tipos dentro de definiciones anidadas y regla de existencia incompleta. En esta versión los tres puntos están corregidos con evidencia de código y de ejecución.

| Punto de rúbrica anterior                              | Corrección aplicada                                                                                                        | Evidencia                                                                     |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Soporte de tipos definidos por usuario.                | Domain acepta nameDomain(ID) y Checker lo convierte a userType(name).                                                      | spec_types.vl usa Person y Group; el caso válido pasa en Rascal y Kotlin.     |
| Correspondencia de tipos.                              | Checker define tipos primitivos, userType, functionType y valida igualdad con TypePal.                                     | Expresiones y operadores con tipos incompatibles fallan en instance/errors/.  |
| Tipos dentro de definiciones anidadas.                 | typedSpaceElementTypes y quantifiedVariableType asignan a la variable cuantificada el tipo de elemento del espacio tipado. | forall member in Group trata member como Person y permite member belongsTo g. |
| Regla de existencia de elementos/tipos en estructuras. | useDomainIfUserDefined registra el uso de dominios definidos por usuario contra spaceId.                                   | spec_unknown_space_error.vl reporta Undefined space Person.                   |

## 8. Evidencia de ejecución y pruebas

Se verificaron pruebas en dos niveles: Rascal por consola y Kotlin/Windows por UI. Las pruebas Rascal también están respaldadas por los logs incluidos en docs/. Las pruebas Kotlin fueron ejecutadas en Windows 11 desde la carpeta kotlin-app.

### 8.1 Pruebas obligatorias en Rascal

| Prueba                                                               | Resultado esperado                                                  | Resultado observado                                                                          | Estado |
|----------------------------------------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------|
| import Main; main();                                                 | Parse OK, checker OK, generación y retorno int: 0.                  | Retorna int: 0 y genera el programa TypeExamples.                                            | Pasó   |
| main( cwd:///instance/errors/spec_unknown_space_error.vl );          | Undefined space Person, sin generación, retorno int: 1.             | Reporta Undefined space Person y retorna int: 1.                                             | Pasó   |
| import RunnerJson;<br>RunnerJson::main(["instance/spec_ty pes.vl"]); | JSON success:true, parseOk:true, typeCheckOk:true, semanticOk:true. | JSON válido con todos los checks en true.                                                    | Pasó   |
| RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);   | JSON success:false y error Undefined space Person.                  | JSON válido con success:false, typeCheckOk:false, semanticOk:false y Undefined space Person. | Pasó   |

Fragmento de evidencia del caso inválido:

```

El programa contiene errores semanticos:
error("Undefined space `Person`", |cwd:///instance/errors/spec_unknown_space_error.vl| (...), fixes=[])
No se genera codigo porque el programa no paso el chequeo semantico.
int: 1

```



## 8.2 Pruebas de todas las instancias incluidas

| Archivo                                                   | Propósito                                                        | Resultado                         |
|-----------------------------------------------------------|------------------------------------------------------------------|-----------------------------------|
| instance/spec1.vl                                         | Caso válido base con espacios, operadores, reglas y expresiones. | Retorna int: 0.                   |
| instance/spec2.vl                                         | Caso válido adicional con subespacios y operadores.              | Retorna int: 0.                   |
| instance/spec3.vl                                         | Caso válido con regla unaria.                                    | Retorna int: 0.                   |
| instance/spec_types.vl                                    | Tipos primitivos y de usuario, Group : Person y cuantificador.   | Retorna int: 0.                   |
| instance/spec_typed_space_good.vl                         | Espacio tipado y relación Element -> Set.                        | Retorna int: 0.                   |
| instance/spec_typed_space_quantifier_good.vl              | Cuantificador sobre espacio tipado.                              | Retorna int: 0.                   |
| instance/errors/spec_unknown_space_error.vl               | Tipo Person usado sin definir.                                   | Falla con Undefined space Person. |
| instance/errors/spec_typed_space_quantifier_type_error.vl | Tipo de variable cuantificada incompatible con operador.         | Falla por error de tipos.         |

## 8.3 Pruebas Kotlin/Windows

Las pruebas Kotlin que faltaban fueron ejecutadas en Windows 11 desde C:

\\Users\\Roni\\PROYECTO\_RASCAL\_P4\\proyecto\_rascal\\kotlin-app.

| #  | Prueba                                  | Resultado observado                                                 | Estado |
|----|-----------------------------------------|---------------------------------------------------------------------|--------|
| 1  | .\\gradlew.bat --version                | Gradle 8.7, Kotlin 1.9.22, JVM 21.0.10, Windows 11.                 | Pasó   |
| 2  | .\\gradlew.bat clean build              | BUILD SUCCESSFUL in 21s; 5 actionable tasks executed.               | Pasó   |
| 3  | .\\gradlew.bat run                      | La aplicación abre correctamente.                                   | Pasó   |
| 4  | Título de ventana                       | La ventana se titula VeriLang y la UI muestra VeriLang — Runner.    | Pasó   |
| 5  | Seleccionar spec_types.vl               | La UI carga el archivo válido.                                      | Pasó   |
| 6  | Validar estados del caso válido         | Parse: OK, Types: OK, Semántica: OK.                                | Pasó   |
| 7  | Ver salida/generación                   | La UI muestra Código generado y Output.                             | Pasó   |
| 8  | Seleccionar spec_unknown_space_error.vl | La UI carga el archivo inválido.                                    | Pasó   |
| 9  | Validar estados del caso inválido       | Parse: OK, Types: FAIL, Semántica: FAIL.                            | Pasó   |
| 10 | Ver error esperado                      | Se muestra Undefined space Person en errores de tipos y semánticos. | Pasó   |

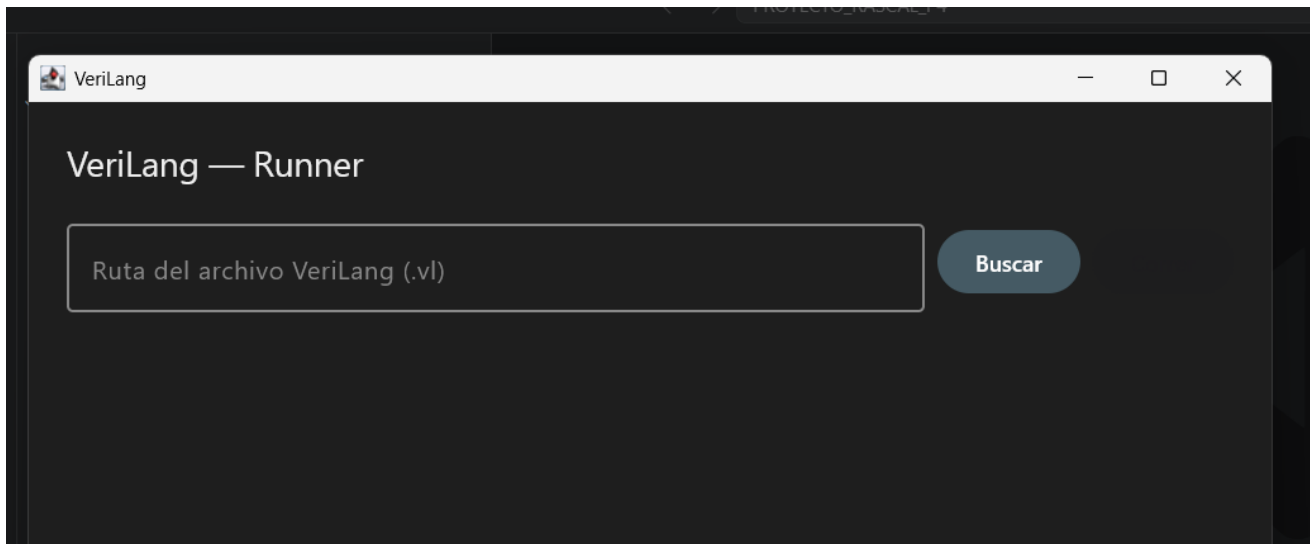


Figura 1. Aplicación Kotlin abierta con título VeriLang y encabezado VeriLang — Runner.

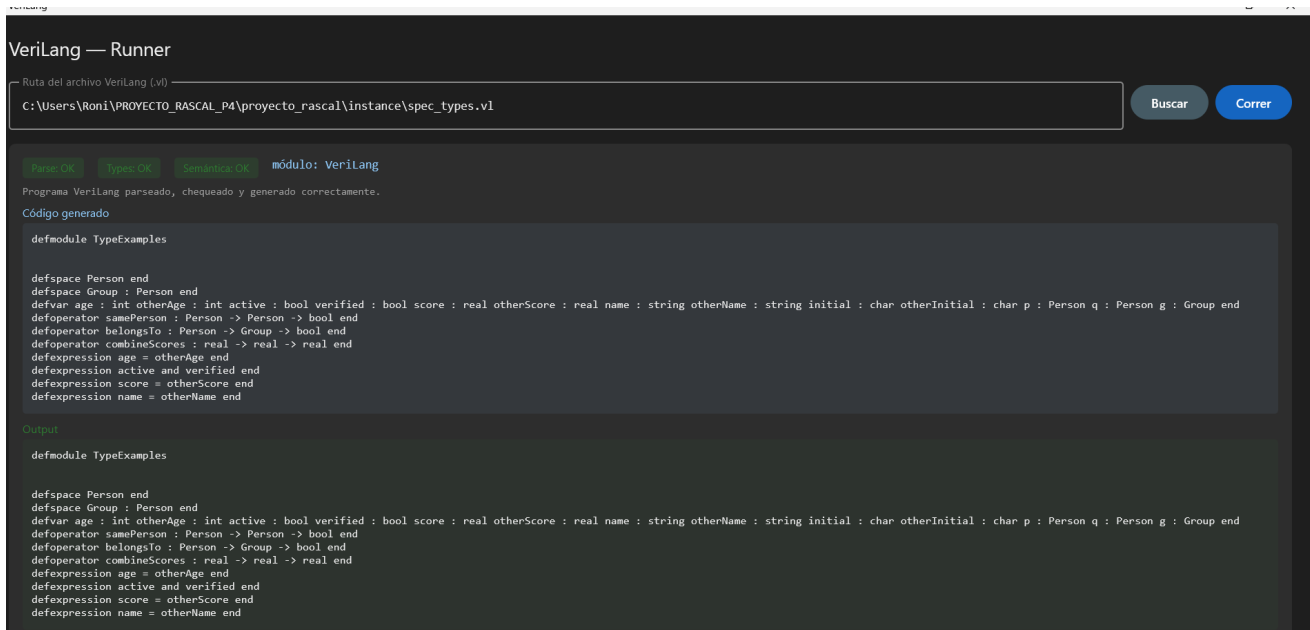


Figura 2. Caso válido spec\_types.vl: Parse OK, Types OK, Semántica OK y código generado.

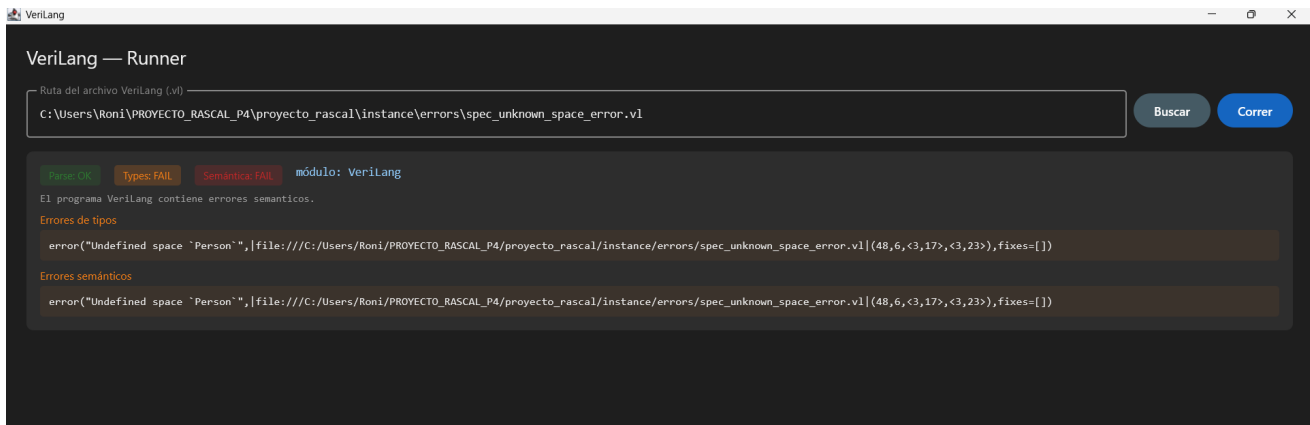


Figura 3. Caso inválido spec\_unknown\_space\_error.vl: Types FAIL, Semántica FAIL y Undefined space Person.

## 9. Matriz de verificación final

| Requisito                         | Evidencia                                                                                | Prueba realizada                              | Estado |
|-----------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------|--------|
| Carpeta raíz y manifiesto         | META-INF/RASCAL.MF declara Project-Name: proyecto_rascal.                                | Inspección y ejecución desde proyecto_rascal. | Cumple |
| Archivos Rascal mínimos           | Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson están presentes.     | Inspección del ZIP.                           | Cumple |
| Sin basura generada               | No se requiere target, build, .gradle, .git, bin ni .lock dentro de la entrega final.    | Inspección de estructura.                     | Cumple |
| Gramática con tipos primitivos    | Domain incluye int, bool, real, string y char.                                           | spec_types.vl.                                | Cumple |
| Tipos definidos por usuario       | Domain incluye nameDomain y Checker usa userType.                                        | spec_types.vl usa Person y Group.             | Cumple |
| Estructuras tipadas               | typedSpace y typedSubspace implementan defspace Group : Person end.                      | spec_typed_space_good.vl y spec_types.vl.     | Cumple |
| Tipos en cuantificadores anidados | typedSpaceItemTypes y quantifiedVariableType asignan el tipo correcto.                   | spec_typed_space_quantifier_good.vl.          | Cumple |
| Regla de existencia               | useDomainIfUserDefined exige que el userType exista como spaceId.                        | spec_unknown_space_error.vl.                  | Cumple |
| Generator                         | Genera texto VeriLang de módulos, espacios, variables, operadores, reglas y expresiones. | main(); y UI válida.                          | Cumple |
| Main.rsc                          | Retorna 0 si no hay errores y 1 si hay errores.                                          | Pruebas obligatorias Rascal.                  | Cumple |
| RunnerJson.rsc                    | Produce JSON válido con success, parseOk, typeCheckOk, semanticOk y errores.             | Pruebas RunnerJson válidas e inválidas.       | Cumple |
| LangService.kt                    | Invoca Rascal, ejecuta RunnerJson, extrae JSON y decodifica RunResult.                   | Ejecución UI en Windows.                      | Cumple |
| RunResult.kt                      | Campos coinciden con el JSON de RunnerJson.                                              | Inspección y UI.                              | Cumple |
| MainWindow.kt                     | Permite seleccionar .vl y muestra Parse/Types/Semántica/errores/output.                  | Capturas de UI.                               | Cumple |
| Kotlin build                      | Gradle wrapper compila el proyecto.                                                      | .\gradlew.bat clean build.                    | Cumple |
| Kotlin ejecución                  | La app abre en Windows y procesa archivos .vl.                                           | .\gradlew.bat run y pruebas UI.               | Cumple |

## 10. Instrucciones de ejecución en Windows

### 10.1 Ejecutar la app Kotlin

```
cd C:\Users\Roni\PROYECTO_RASCAL_P4\proyecto_rascal\kotlin-app
.\gradlew.bat --version
.\gradlew.bat clean build
.\gradlew.bat run
```

Después de abrir la ventana, usar Buscar para seleccionar un archivo .vl. Para el caso válido se recomienda instance\spec\_types.vl. Para el caso inválido se recomienda instance\errors\spec\_unknown\_space\_error.vl.

### 10.2 Ejecutar Rascal por consola

```
cd C:\Users\Roni\PROYECTO_RASCAL_P4\proyecto_rascal
java -Dfile.encoding=UTF-8 -jar rascal-shell-stable.jar

import Main;
```

```
main();
main(|cwd:///instance/errors/spec_unknown_space_error.vl|);

import RunnerJson;
RunnerJson::main(["instance/spec_types.vl"]);
RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);
```

### 10.3 Nota sobre dependencias

La primera ejecución de Gradle puede requerir internet para descargar dependencias de Gradle/Kotlin/Compose. Una vez descargadas, el build puede reutilizar el caché local. Rascal se ejecuta con rascal-shell-stable.jar incluido en el proyecto.

## 11. Conclusión

La entrega cumple el objetivo del Proyecto 4: reutiliza VeriLang implementado en Rascal y lo ejecuta desde una aplicación Kotlin. La equivalencia funcional con Proyecto 3 queda respaldada porque la app Kotlin no sustituye la semántica, sino que invoca RunnerJson.rsc, que a su vez ejecuta Parser, Checker/TypePal y Generator.

Las fallas señaladas en la retroalimentación anterior fueron corregidas: la gramática y el AST soportan tipos definidos por usuario, el checker valida esos tipos contra espacios existentes, los cuantificadores sobre estructuras tipadas reciben el tipo del elemento y el caso de error Undefined space Person falla correctamente tanto en Rascal como en la UI Kotlin.

Con base en la inspección del código y las pruebas ejecutadas, la solución es entregable. Los riesgos restantes son menores y están principalmente relacionados con el empaque final del ZIP y con asegurar que el documento entregado sea esta versión corregida, sin afirmaciones falsas sobre dependencias Maven de TypePal ni rutas de paquete antiguas como milang.